GreenTwin

System design and decoupled build blocks

Megathon X: Singularity · Lloyds Problem Statement 2: Green Home Score and Retrofit Recommendation Engine · Team MegaMind

> **In one line.** A Bayesian-calibrated physics twin of every home, which you can see in 3D, explain in plain language and act on, where every number is backed by physics and the range it comes with.

Audience: the build team. This document fixes the architecture, the contracts between parts, and eight build blocks that can each be built, tested and documented independently. Design detail for individual techniques lives in docs/STRATEGY_REPORT.md; binding rules live in AGENTS.md and contracts/, which win over this document if they ever disagree.

Section	What it covers
1. Philosophies	The six principles every block follows
2. Architecture	End-to-end system diagram, layers and the what-if loop
3. Uncertainty	How every number becomes a full distribution
4. Build blocks	Eight decoupled blocks, their interfaces and dependency graph
5. Block specs	Goal, build steps, models, outputs and done-when for each block
6. Key mechanisms	Retrofit engine + improvement ladder, energy prediction, property value
7. Evaluation & submission	Metrics vs baselines, PS mapping, submission items A–I
8. Working rules	Hackathon coding and UI guidelines, open decisions

1. Core philosophies

#	Philosophy	What it means in the system
1	**Physics is the source of truth**	Every energy, savings, carbon and rating number comes from the Level-1 digital twin. ML sets priors (EBM), estimates value and checks results; it never produces a customer's savings, so the system handles retrofit combinations no dataset has seen.
2	**Bayesian in thought**	Each home is calibrated against its own meter readings. Every output is a full distribution (mean, median, mode, std, P1–P99, 90% interval, tail probabilities) plus P(Poor), P(suitable), P(on front). Missing data never becomes zero; it widens the range.
3	**Rerun, never add up**	A retrofit is a change to the twin's parameters; every what-if and every package reruns the model. Interactions (insulation before a heat pump, electricity rising as gas falls) come from the physics. Savings are always per fuel.
4	**Grounded explanation**	The LLM can only call tools and quote what they return; a validator rejects any other number. Jev only routes questions and screens feasibility. No backing → "unavailable", never a guess.
5	**Explainability first**	Score waterfall, heat loss per building element, EBM contributions, value-model coefficients, and a reason on every recommendation or veto.
6	**See it to believe it**	A living 3D twin explored inside and out: heat loss glows, retrofits appear as they are added, and score, savings and bills update live, all from the same calibrated model.

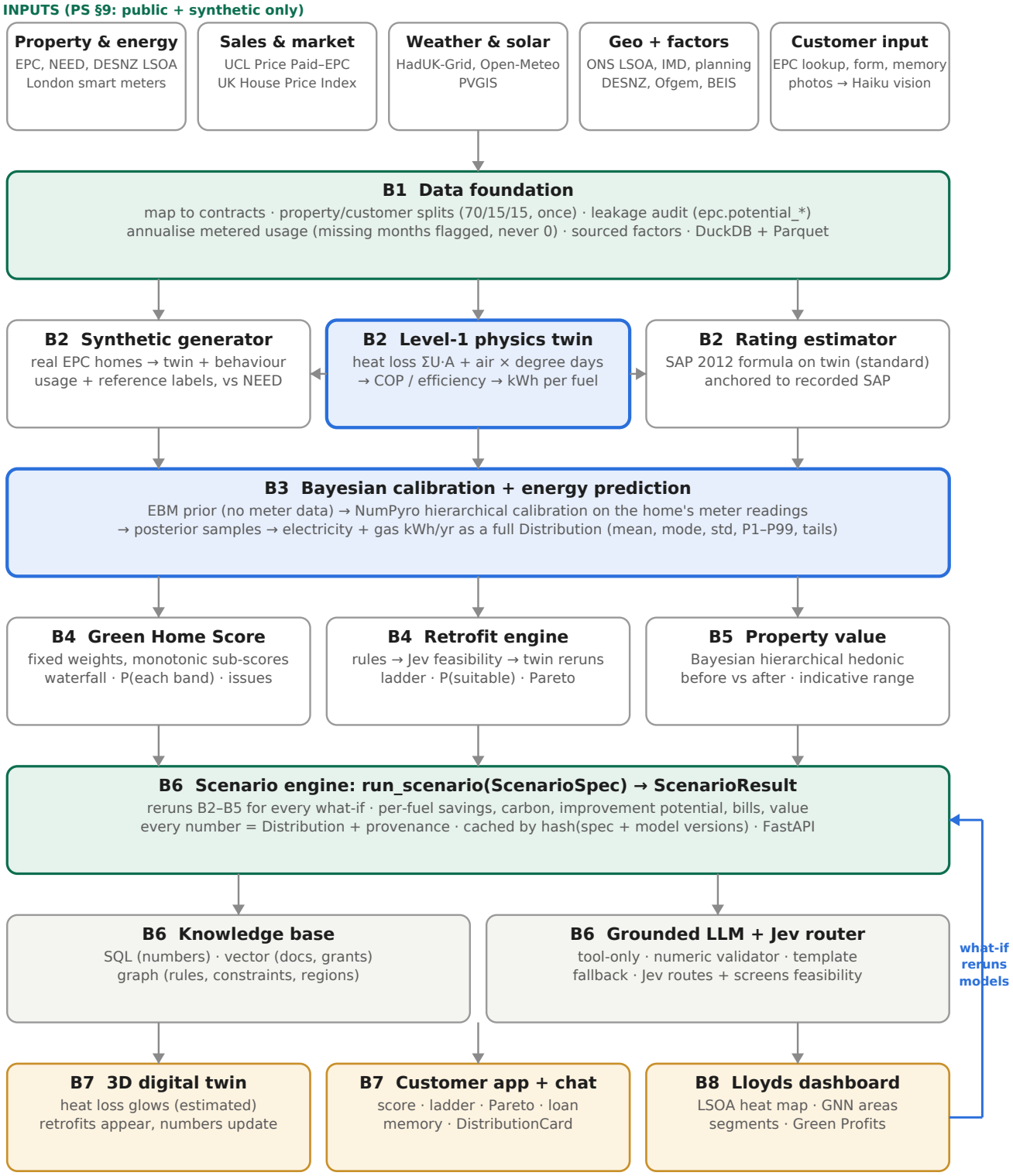
The core rule from the brief, and how the design enforces it

"*The score, savings, carbon and value figures come from documented models and calculations. The LLM only explains them... For a what-if request, the relevant models must first be rerun.*" (PS §1)

- Only **B2/B3** (physics + Bayesian), **B4** (score formula, rules) and **B5** (value model) produce numbers. Each number carries provenance naming its model.
- Every what-if goes through **run_scenario** (B6) before the LLM sees anything.
- The LLM's answer passes a **numeric validator**: any number not in that turn's tool output is rejected → regenerate once → template explainer.

2. System architecture (end to end)

Data flows top to bottom through one physics twin. Everything below the twin consumes its posterior samples; the experience layer never computes, it renders ScenarioResult. A what-if from the 3D twin or the chat loops back into the scenario engine (blue).

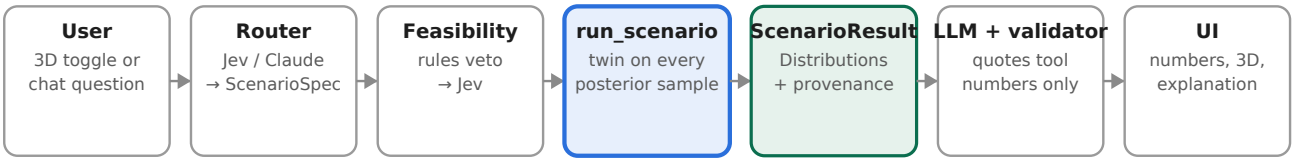


Colour key: blue = physics/Bayesian core (the only source of energy numbers); green = data and orchestration; grey panels = LLM side (reads only through tools); amber = what users see.

Layers and their contracts

Layer	Blocks	Consumes	Produces (contracts/)
Inputs + data	B1	public datasets, customer input	Property, EnergyUsageMonth, SaleRecord, WeatherMonth, factor tables, retrofit catalogue
Physics	B2	Property, WeatherMonth, catalogue	TwinParams, TwinOutput (per-fuel kWh, ElementHeatLoss), RatingEstimate, synthetic data + labels
Bayesian + prediction	B3	TwinParams, usage, weather	posterior samples; EnergyBreakdown (Distributions)
Decisions + value	B4, B5	posterior samples, Property, sales	ScoreResult, Feasibility, RetrofitEffect, ParetoPoint, ValueImpact
Orchestration	B6	all of the above	ScenarioResult (one per what-if), API, chat answers, WhatIfRoute, ExplanationCheck, CustomerMemory
Experience	B7, B8	API + contracts/types.ts	customer app, 3D twin, Lloyds dashboard

The what-if loop



Low router confidence → clarifying question. Validator failure → regenerate once → template explainer. Missing data → "unavailable" + reason.

Technology choices

Concern	Choice
Data + storage	Python 3.12, pandas/Polars, DuckDB + Parquet (with meta.json per file)
Physics twin	**JAX** (jax.numpy): the twin runs inside the NumPyro model, so NUTS/SVI get exact gradients, and jax.vmap runs it over all posterior samples at once. Monthly steady-state RC model first; differentiable RC graph (diffrax ODE) later behind the same simulate()
Electricity layer	Half-hourly profile per home: London smart-meter load-shape archetypes scaled to annual kWh + heat pump (degree-hours) + PVGIS hourly PV − battery dispatch → self-consumption, time-of-use bills
Bayesian	NumPyro: NUTS offline for hierarchy, SVI per home (< 1 s); ArviZ diagnostics
Learned models	EBM (InterpretML) primary; XGBoost baseline; GraphSAGE GNN (PyTorch Geometric) on the LSOA graph
Photo-to-twin	Claude Haiku 5.5 (claude-haiku-5-5) vision with structured outputs: floor plan → rooms, room photos → element tags with confidence; react-konva floor-plan editor for agent correction
Value	NumPyro hierarchical hedonic regression; OLS baseline
API	FastAPI; run_scenario cached by hash(spec + model versions)
LLM	Claude tool use + numeric validator + template fallback; Jev (System 1) if a key exists, else Claude structured output
Front end	React + Vite + TypeScript, react-three-fiber + drei (3D), Recharts, MapLibre + deck.gl, Zustand

3. Uncertainty: every number is a distribution

We never know a home's exact physics, so we keep many plausible versions of it (posterior samples) and run everything on all of them. Each output is computed **once per sample**, then summarised. Intervals are never added or multiplied together: the samples carry the correlations (a leaky home saves more from every retrofit).

The Distribution contract (contracts/models.py, built by contracts/summarise.py)

Part	Fields	Why
Centre	mean · median · mode (peak of the sample density)	They differ when the result is lopsided; showing all three stops the mean hiding it
Spread	standard deviation · IQR	How uncertain the number is
Percentiles	P1, P5, P10, P25, P50, P75, P90, P95, P99	The full shape, tails included
Credible intervals	50% (P25–P75) and 90% (P5–P95)	The 90% interval is the headline range
Tails	labelled tail probabilities (e.g. P(bill rises > £100/yr)) · mean of the worst 5% · skewness	What could go wrong, and how badly
Shape + meta	histogram · unit · n_samples · headline (mean or median if skewed)	Density plot in the UI; traceability

Categories (score band, EPC band, verdict) report the probability of every category plus the mode. Probabilities (P(suitable), P(on front)) are reported as they are. Only documented constants (PS weights, emission factors, tariffs, grants, rule vetoes) are bare numbers.

Where the uncertainty comes from

Source	How it is modelled
The home's physics (heat loss, setpoint, baseload, COP)	Posterior from calibration on its meter readings; prior only (wider) without meter data
Missing months, estimated reads, imputed fields	Latent variables / wider noise, never 0
Weather	Typical, cold and warm years sampled
Rebound after retrofit	Prior on setpoint rising after fabric retrofits
Retrofit costs	Distribution over the sourced low–high range (BEIS 2017 × CPI)
Rating after retrofit	Twin samples through the SAP estimator → SAP distribution + band probabilities
Property value	Posterior of the hedonic model (location effects, rating effect)
Feasibility	Jev confidence recalibrated (isotonic) into P(feasible); rule veto = 0

Example: heat-pump bill change on SH2 (illustrative)

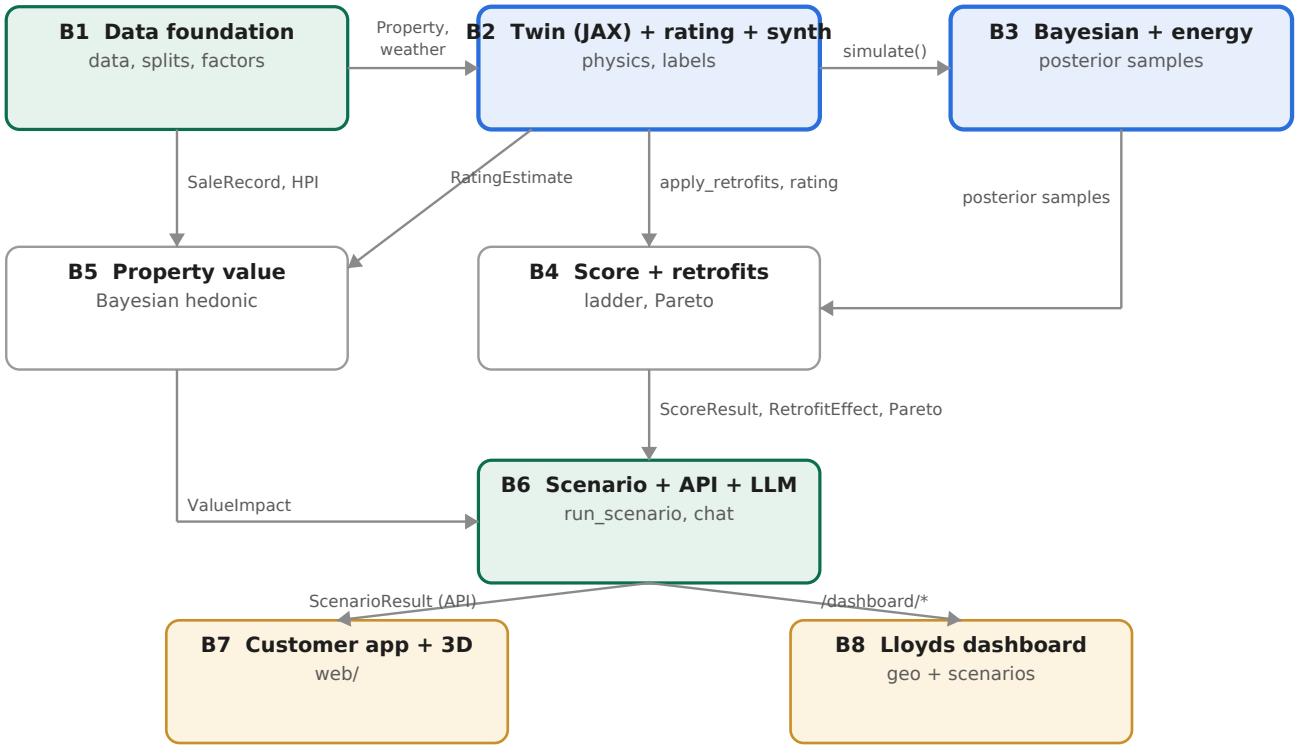
Mean	Median	Mode	Std	P5	P25	P75	P95	P(saving > 0)	P(rise > £100)
−£35	−£42	−£55	£88	−£180	−£95	+£20	+£110	0.38	0.23

The mean alone hides that about 1 in 4 homes like this could see the bill rise by over £100/yr, while carbon falls with near certainty. That is the brief's "electricity rises after a heating change" scenario, stated honestly.

4. Build blocks (decoupled parts of the system)

Eight blocks, each a complete piece one person can build, test and document end to end. Blocks talk **only** through contracts/ (Pydantic models + the TypeScript mirror), so every block starts on day one against contracts/examples/ and stubs.

Arrows = contracts passed between blocks; every block can start on mocks.



Interfaces: what each block exposes

Block	Package	Public interface (stub it first)	Start on
B1 Data	greentwin/data, geo	load_property(id), load_usage(id), load_weather(area), load_sales(), emission_factors(), tariffs(), catalogue()	examples + small CSV samples
B2 Twin	greentwin/twin	params_from_property(p), simulate(params, weather, mode), apply_retrofits(params, actions), estimate_rating(...), electricity_profile(p, params) (half-hourly), generate_synthetic(n, seed)	RdSAP defaults
B3 Bayesian	greentwin/bayes, models	intensity_prior(p) → Distribution, calibrate(p, usage, weather) → Posterior, predict(posterior, spec) → samples	synthetic data from B2
B4 Score + retrofits	greentwin/score, retrofit, optimise	score(p, energy) → ScoreResult, feasibility(p) → [Feasibility], ladder(p, posterior, goal) → [RetrofitEffect], pareto(...) → [ParetoPoint]	recorded inputs; fake samples
B5 Value	greentwin/value	value_impact(p, rating_after) → ValueImpact, green_twins(p)	UCL sales sample
B6 Orchestration	greentwin/api, llm, jev, kb	run_scenario(spec) → ScenarioResult; REST API; chat(...), route(...) → WhatIfRoute; extract_floorplan(image), tag_photo(image) → draft Property fields + confidence	MOCK ScenarioResult
B7 Customer app	web/	screens over the API using contracts/types.ts; react-konva floor-plan editor	mock server
B8 Dashboard	greentwin/geo + web/	area_layers(layer, region), segments(), green_profits(assumptions), lsoa_gnn() → expected kWh per LSOA (Distribution)	B1 data + batch outputs

5. Block specifications

B1 · Data foundation

PS §9 · feeds B2, B3, B4, B5, B8

Goal. Clean, licensed, split, joinable real data with every external factor traceable to a cited source.

- uv + Python 3.12 environment; CI runs pytest.
- Download scripts → Parquet + meta.json: EPC certificates + recommendations + certificate history (North West, South West, London), NEED 50k, UCL Price Paid–EPC linked sales + UK HPI, weather degree days (HadUK-Grid / Open-Meteo), ONS postcode → LSOA, IMD, off-gas, planning constraints, OSM roofs, **London smart meters** (half-hourly, 5,567 homes; load shapes) and **DESNZ sub-national LSOA electricity + gas consumption** (real area-level labels).
- Sourced factors in contracts/: DESNZ 2025 emissions, Ofgem tariffs, BEIS 2017 × CPI costs; unsourced items return null + reason.
- Mappers to Property / SaleRecord; splits.py (70/15/15 by customer, else property, seed 42, assigned once); leakage audit (epc.potential_* are labels only); load into DuckDB.

Outputs. Real data tables, factor tables, load_* helpers, docs/DATA.md

Done when. No id spans two splits; no leakage fields in feature lists; every table has meta.json; DATA.md done.

B2 · Physics twin + rating estimator + synthetic data

PS §5, §7, §9 · feeds B3, B4, B5, B7

Goal. A documented physics model that turns any home and retrofit package into per-fuel energy and an estimated rating, and a generator that uses it to create realistic training data and labels.

- Level-1 twin **in JAX**: element U·A from RdSAP defaults (room geometry when photo-to-twin provides it), ventilation, degree days, hot water, appliances; boiler efficiency / heat-pump COP → electricity + gas; PV cuts imports. Modes: calibrated and standard (SAP occupancy).
- Electricity layer: half-hourly profile from smart-meter load-shape archetypes + heat pump + PVGIS PV − battery dispatch, giving self-consumption and time-of-use bills; lighting share for LED retrofits.
- apply_retrofits: each catalogue item is a parameter edit; packages are always re-simulated.
- Rating estimator (contracts/sap_estimator.json): SAP 2012 formula on the twin in standard mode, anchored to the recorded SAP; labelled "estimated rating, not an EPC".
- Synthetic generator: real EPC homes + NEED-calibrated behaviour noise → monthly usage with gaps and conflicts, plus reference labels (per-fuel savings, improvement potential, feasibility); planted value premium for method checks only.

Outputs. TwinParams, TwinOutput, ElementHeatLoss (estimated=true), RatingEstimate, synthetic datasets

Done when. SH1–SH3 within ±15% of metered totals; monotonicity sweeps pass; synthetic vs NEED report; rating validated on EPC before/after pairs.

B3 · Bayesian calibration + energy prediction

PS §3, §5, §6 · feeds B4, B6

Goal. Fit each home's unknown physics to its own meter data and predict per-fuel energy with full distributions.

- EBM energy-intensity prior trained on NEED (baselines: NEED median, XGBoost).
- NumPyro hierarchical calibration (home → LSOA → region, by type) of heat loss, setpoint, baseload, COP; missing months latent; NUTS offline, SVI per home.
- predict(): every posterior sample through apply_retrofits + simulate for baseline, package and each step in sequence; weather years sampled; rebound prior.
- Annualised metered intensity for the score (not weather-normalised); weather-normalised figure as a sensitivity only.

Outputs. Posterior samples, EnergyBreakdown of Distributions, data-confidence score

Done when. Held-out 90% coverage ≈ 90%, CRPS, R-hat < 1.01; beats degree-day OLS / LightGBM quantile; masked-month imputation error reported.

B4 · Green Home Score + retrofit engine

PS §2, §3, §4 · feeds B6, B7, B8

Goal. Answer "how green is my home" and "what should I improve" with feasible, ordered, explained recommendations.

- Score from contracts/score_mapping.json: fixed weights, monotonic sub-scores, rounded, waterfall; score distribution and P(each band) from posterior samples; overlaps justified.
- Poor-home detector: P(Poor/Very poor), key issues per m² with peer context.
- Feasibility: rule vetoes → Jev (or Claude fallback) on survivors → calibrated P(feasible).
- Improvement ladder: twin reruns in fabric-first order; per-step distributions; P(suitable) = P(feasible) × P(net benefit) → High/Medium/Low; probabilistic verdicts.
- Electricity retrofits: battery (self-consumption, time-of-use shifting), LED lighting, time-of-use tariff (changes bills, not kWh; reported as such).
- Pareto front over feasible packages with P(on front), knee and budget filter.

Outputs. ScoreResult, Feasibility, RetrofitEffect, ParetoPoint, docs/SCORE.md

Done when. SH1 blocks EWI; SH2 shows electricity rising; SH3 blocks loft/PV and marks the heat pump needs_survey; ranking vs reference labels and vs real EPC recommendations reported.

B5 · Property value

PS §7 · feeds B6, B7

Goal. An indicative value range, before vs after, accounting for location, size, type and market.

- Bayesian hierarchical hedonic model on real sales: log(price ÷ HPI) = location (LSOA within LA within region) + size + type + form + age + tenure + β(region × type) × rating.
- Same home at recorded vs estimated-after rating, applied to the current value estimate (never a past sale price); full distribution; wording always "indicative".
- Stretch checks: repeat sales, matched comparable homes ("green twins of your home").

Outputs. ValueImpact (Distribution + wording), docs/VALUE.md

Done when. Held-out coverage and error vs OLS without the rating; β vs Nationwide / DECC 2013; planted-premium recovery reported as a method check only.

B6 · Scenario engine + API + knowledge base + grounded LLM + memory

PS §1, §8, §10 · feeds B7, B8

Goal. One run_scenario that reruns every model, an API over it, and an LLM that only explains returned numbers.

- run_scenario assembles ScenarioResult from B2–B5 with provenance; caching; mock mode for the UI.
- FastAPI endpoints: homes, scenario, feasibility, pareto, green-twins, chat, route, dashboard, memory.
- Knowledge base: SQL (numbers), vector store (docs, grants), embedded graph (rules, constraints).
- LLM: tools only, system-prompt rules, numeric validator, template fallback, cached calls; Jev router with isotonic calibration, Claude fallback.
- Customer memory updates constraints and posteriors; ≥ 30 adversarial LLM scenarios.

Outputs. ScenarioResult, REST API, chat answers with ExplanationCheck, docs/LLM.md

Done when. Real results for SH1–SH3 validate; 0 unvalidated numbers; groundedness and accuracy ≥ 90%, unsupported claims < 5%; works with no keys.

B7 · Customer app + 3D twin

PS §2–§7, §10 · the demo

Goal. The customer journey from "how green is my home" to a green-loan conversation.

- Shared theme + components (NumberWithRange, DistributionCard, ScoreWaterfall, ProvenanceTag).
- Screens: score + waterfall, improvement ladder, what-if chat, 3D twin, Pareto + green loan, memory, agent capture.
- Photo-to-twin: upload floor plan + room photos → Haiku vision drafts rooms and element tags with confidence → agent corrects in a react-konva editor → confirmed fields fill Property and room-level geometry drives the 3D house. v1 fallback: photos + manual tags.
- 3D: parametric house from EPC fields; outside, cutaway and thermal views; retrofits appear; numbers update via run_scenario; precomputed packages for showcase homes.

Outputs. web/ app

Done when. Full journey runs live on SH1–SH3 and offline from cache; demo script runs without errors.

B8 · Lloyds dashboard + geography

PS §12 business relevance · the business story

Goal. Turn home-level results into green-lending demand; every number computed or a labelled assumption.

- Batch scoring over EPC regions → LSOA / district aggregates (Distributions summed over samples).
- Heat map layers: green share, Poor share, finance demand, carbon potential, grant eligibility, hotspots, degree-day trends.
- Segments; Green Profits scenario from contracts/green_profits_assumptions.json (assumption sliders, Monte Carlo); funnel. Synthetic customer base labelled as such.
- GNN area model: GraphSAGE on the LSOA adjacency graph (features: EPC stock mix, IMD, weather, off-gas share) predicting DESNZ mean domestic electricity and gas per meter; uses: area priors for the Bayesian hierarchy, an "uses more than expected" map layer, filling suppressed LSOAs.

Outputs. /dashboard/* API + dashboard screens

Done when. Runs on three regions; every assumption visibly labelled.

6. Key mechanisms

6.1 Retrofit engine: rules + Jev decide what is possible; the twin decides what it is worth

- **Feasibility:** hard rule vetoes first (conservation area → no external wall insulation, no roof access → no PV/loft, ...), then Jev on the survivors: feasible / with conditions / needs survey / infeasible with a calibrated confidence. Jev never produces a number.
- **Simulation:** step k's marginal effect = twin(steps 1..k) − twin(steps 1..k−1) on every posterior sample, in the order fabric → glazing → controls → heating → generation. The marginals add up exactly to the package, and the package is one twin run.
- **Verdicts (probabilistic):** Recommended if P(suitable) ≥ 0.8 and P(step adds ≥ 3% of purchased energy or ≥ 2 score points) ≥ 0.8; Optional / Not worth adding / Needs survey / Not feasible otherwise. Thresholds to be agreed; "net benefit" follows the customer's goal (bills or carbon).

Improvement ladder, SH2 (illustrative; mean with 90% interval). Baseline 15,200 kWh/yr purchased, score 60.

#	Add	Gas saving kWh/yr	Elec. saving kWh/yr	Potential so far	Score after	CO2 t/yr	Bill £/yr	Verdict
1	Cavity wall insulation	2,600 (2,100–3,100)	0	17%	66	0.48	+207	Recommended
2	Loft top-up to 270 mm	900 (650–1,150)	0	23%	68	0.16	+72	Recommended
3	Draught proofing	400 (200–600)	0	26%	69	0.07	+32	Optional
4	Smart thermostat + TRVs	500 (300–750)	0	29%	71	0.09	+40	Recommended
5	Replace old double glazing	450 (250–650)	0	32%	72	0.08	+36	Not worth adding
6	Air-source heat pump	7,150 (6,500–7,800)	−2,300 (rises)	64%	80	0.86	−35	Depends on goal
7	Solar PV 3 kWp	0	1,100 (import cut)	71%	86	0.22	+290	Recommended
	Package (one run)	**12,000**	**−1,200**	**71%**	**86**	**1.96**	**+641**	

Carbon: DESNZ 2025 (gas 0.18296, electricity 0.19553 kgCO2e/kWh). Bills: Ofgem Q4 2026 (gas 7.97p, electricity 26.32p). The heat pump raises electricity and leaves the bill centred near zero at current prices; removing the gas supply also saves the ~£108/yr standing charge, reported separately.

6.2 Improvement potential (PS §3)

- 100 × (E_baseline − E_after) / E_baseline on purchased energy (grid import + gas), E_after from **all feasible retrofits** run together; reported as a distribution with the per-fuel split.
- Needs-survey items are reported separately; potential (technical maximum) is not the customer's plan.
- Baseline 0 or missing → unavailable + reason. Homes are ranked by the posterior mean with P(top 10%).

6.3 Energy prediction (B2 + B3)

- Physics computes heat loss × degree days → fuel via efficiency / COP; solar cuts imports.
- Homes without meter data: EBM prior (trained on NEED) sets the expected kWh/m², wider distribution.
- Homes with meter data: Bayesian calibration fits the twin to the home's own bills.
- Output: electricity and gas kWh/yr as full Distributions; checked on held-out homes against the NEED median and XGBoost.

6.4 Property value (B5)

- log(price ÷ local HPI) = location (LSOA within LA within region, pooled) + floor area + type + form + age + tenure + β(region × type) × rating.
- Value impact per posterior sample = V × (exp(β × ΔSAP) − 1), with ΔSAP from the rating estimator → full distribution; if the range includes 0, say the data shows no clear effect.
- Always "Indicative range based on historical associations in sales data; not a promised or proven increase." Never a reason to borrow.

6.5 Photo-to-twin (agent capture, B6 + B7)

- **Flow:** agent or customer uploads a floor-plan photo and room photos → **Claude Haiku 5.5** (claude-haiku-5-5) vision with structured outputs returns rooms (label, polygon, estimated dimensions) and element tags (glazing type, boiler/heat pump, radiators, visible loft depth, ...) each with a confidence → the agent corrects polygons and tags in a **react-konva** editor → confirmed fields fill Property.
- **What room geometry is for:** floor area cross-checked against the EPC, wall lengths and window areas for the twin's U·A areas, and real rooms in the 3D house. The physics stays home-level: monthly bills cannot identify per-room parameters.
- **Guardrails:** vision outputs are tags with confidence, never energy numbers; low confidence goes back to the agent; every field needs agent confirmation. Consent, faces and personal items blurred, stated retention. v1 fallback: photos + manual tags.
- **Evaluation:** room-area error and tag accuracy on a labelled set (CubiCasa5k floor plans + our photos), confidence calibration, share of fields the agent had to correct.

6.6 Electricity depth (B1 + B2 + B4 + B8)

- **Data:** London smart meters (half-hourly, 5,567 homes, 2011–14) → load-shape archetypes (clustered daily profiles by season and day type); DESNZ sub-national LSOA consumption → real area-level electricity and gas per meter.
- **Twin electricity layer:** each home's electricity = archetype profile scaled to its calibrated baseload + heat-pump demand from degree-hours + PVGIS hourly PV − battery dispatch. Gives PV self-consumption, export, and time-of-use bills; monthly totals stay consistent with the monthly twin.
- **Electricity retrofits:** battery (shifts PV and cheap-rate electricity), LED lighting (cuts the lighting share of baseload), time-of-use tariff (changes bills, not kWh, and is labelled that way). Time-of-use rates need a cited supplier source before any bill figure is shown.
- **GNN on LSOAs:** GraphSAGE over the LSOA adjacency graph predicts DESNZ mean consumption from stock mix, IMD, weather and off-gas share. Baselines: EBM/LightGBM without the graph; spatial-block CV to avoid leakage. Uses: priors for the Bayesian hierarchy, an "uses more than expected" dashboard layer, filling suppressed areas.

7. Evaluation and submission

Evaluation (held-out, property-level splits, simple baselines)

Output	Metric	Baseline	Block
Energy intensity / prediction	MAE/MAPE per fuel, 90% coverage	NEED median, XGBoost	B3
Calibrated per-fuel energy	coverage ≈ 90%, CRPS, PIT, R-hat	degree-day OLS, LightGBM quantile	B3
Missing-month imputation	MAE on masked months	seasonal-naive ×12	B3
Rating after retrofit	MAE + coverage of SAP change on EPC before/after pairs	median band shift per measure	B2
Score	0 monotonicity violations; every heating type mapped	—	B4
Improvement-potential ranking	Spearman / NDCG vs reference labels	rank by total kWh	B4
Recommendations	precision@k vs real EPC assessor lists; 0 infeasible	recommend all feasible	B4
Pareto	hypervolume, % greedy picks dominated	greedy kWh/£	B4
Value	coverage, pinball loss on held-out sales; β vs published	OLS without rating	B5
Probabilities	calibration (a "0.8" happens ~80% of the time)	—	B3, B4, B6
LLM	groundedness ≥ 90%, accuracy ≥ 90%, unsupported < 5%	no validator	B6
Synthetic data	KS / quantiles vs NEED	—	B2
GNN area model	MAE per fuel on held-out LSOAs (spatial-block CV)	EBM / LightGBM without graph	B8
Photo-to-twin	room-area error, tag accuracy, confidence calibration, % fields corrected	manual tagging	B6 + B7
Electricity layer	half-hourly profile error vs held-out smart-meter homes; PV self-consumption	flat profile	B2

Problem statement coverage

PS section	Requirement	Block
§1	Core rule; what-if reruns the models first	B6
§2	Score: fixed weights, monotonic documented sub-scores, contributions, justified overlaps	B4
§3	Poor homes, key issues, improvement potential (per fuel)	B3 + B4
§4	Feasible, prioritised retrofits with suitability, saving, reason; catalogue documented	B1 + B4
§5	Per-retrofit and package savings per fuel, interactions, consistent baseline	B2 + B3 + B4
§6	Carbon from documented per-fuel factors	B1 + B3
§7	Indicative value range, before vs after, location/size/type/market	B2 + B5
§8	Expected outputs, pipeline, what-if flow	B6 + B7
§9	Data, schema, splits, leakage, documented synthetic labels	B1 + B2
§10	Customer memory changes recommendations	B6 + B7
§11	Evaluation vs baselines, scenario tests, performance	all

Submission items (PS §13)

Item	Content	Block
A	Working prototype	B7 + B6
B	Source, data scripts, run instructions	all
C	Trained models + inference code (artifacts/)	B2, B3, B4, B5
D	Sub-score mappings (docs/SCORE.md)	B4
E	LLM prompts, model/API, evaluation (docs/LLM.md)	B6
F	Architecture diagram (this document)	B6 + B7
G	Evaluation report on the validation set	all
H	Responsible AI note	B6 + B1
I	Held-out test predictions + schema/label docs	B1 + B3/B4/B5

8. Working rules

Coding guidelines: speed first

> **This is a hackathon.** Speed is the goal, not bug-free code. A working, good-looking demo beats a perfect module nobody sees.

- Build the thinnest end-to-end path first (SH1–SH3 through twin → score → retrofits → API → UI), then deepen. Never polish one block in isolation.
- Work against contracts/examples and mocks from day one; never wait for another block.
- Reuse libraries and code; copy-paste is fine; no premature abstractions. Hardcode and precompute where it helps the demo; cache every slow or paid call.
- Stuck for ~45 minutes → cut scope, use the fallback or ask. Commit small and often; keep the demo path working.
- Tests only where judges look: contracts validate, monotonic sub-scores, per-fuel signs, null improvement potential at baseline 0, the LLM validator, no mock in the demo.

Non-negotiables, even when rushing

- Contracts in and out; provenance on every result; uncertain numbers are Distributions built with contracts.summarise.
- Missing ≠ 0; savings per fuel; packages rerun, never summed; the LLM never produces a number; value always "indicative"; no keys in git.

UI guidelines: one look everywhere

- One design system (web/src/theme) and shared components: NumberWithRange, DistributionCard, ScoreWaterfall, ProvenanceTag, StatusBadge. Never restyle per page.
- Palette: green #0b6e4f primary, blue #2a6fdb accent, neutral greys; colour-blind-safe scales (viridis/cividis) for heat loss and maps; red only for increases and negative savings.
- One font family (Inter), 8 px spacing grid, cards, one chart style; the 3D twin uses the same palette.
- Every number: unit + headline + 90% range; tap → DistributionCard. Labels: estimate, projected, indicative, estimated from home type.
- Polish the demo journey first (score → ladder → 3D → loan → chat); works offline for showcase homes.

Open decisions

Decision	Options	Recommendation
Twin ambition (library decided: JAX)	A: monthly steady-state twin in JAX first · B: differentiable RC graph in JAX (diffrax) first	A for checkpoint 1; upgrade behind simulate() if on time
Verdict thresholds	gain threshold (3% / 2 points), probability cut-offs, cost per kWh cut-off	Agree once; record in docs/SCORE.md
Default goal	bills or carbon for "net benefit"	Bills by default; carbon toggle

Checkpoints

Checkpoint	Done when
1. End-to-end skeleton	SH1–SH3 flow through real twin → score → retrofits → API → UI
2. Complete submission	Bayesian distributions, rating estimator, value, grounded LLM, evaluation report, docs D–I
3. Demo build (freeze)	3D twin, Pareto + loan, Lloyds dashboard, memory